

Cuaderno de Estudio: Control de Flujo

Python 3 — Capítulo 4 | Resumen de Conceptos Clave y Sintaxis

01 Estructura y Reglas Generales de Código

- **Flujo Secuencial:** Por defecto, un programa ejecuta sus instrucciones en orden de arriba a abajo.
- **Indentación (Bloques de código):** Python **no usa llaves** (`{ }`) para definir bloques. En su lugar, utiliza espacios en blanco (la norma PEP 8 recomienda **4 espacios**).
- **Comentarios:** Comienzan con el símbolo almohadilla `#`. Sirven para explicar el código no explícito o aportar contexto, no para duplicar lo que el código ya dice.
- **Ancho de línea:** Se aconseja un límite máximo de **80 caracteres** por línea. Para romper una línea larga se puede usar la barra invertida `\` o encapsular la expresión entre paréntesis `(...)`.

02 Sentencias Condicionales

Permiten bifurcar el flujo de ejecución en función de la veracidad de una condición determinada.

A. Estructura `if`, `elif` y `else`

- `if`: Evalúa una condición booleana principal.
- `else`: Bloque opcional que se ejecuta si la condición principal es falsa.
- `elif`: Combinación de `else + if`. Permite encadenar múltiples condiciones evitando un anidamiento excesivo.

B. Asignación Condicional (Operador Ternario)

Sintaxis simplificada en una sola línea para asignar valores a variables:

```
variable = valor_si_verdadero if condicion else valor_si_falso
```

C. Operadores de Comparación y Lógicos

Categoría	Operador / Símbolo	Descripción
Comparación	<code>==</code> , <code>!=</code>	Igualdad y Desigualdad
	<code><</code> , <code><=</code>	Menor que, Menor o igual que
	<code>></code> , <code>>=</code>	Mayor que, Mayor o igual que
Lógicos	<code>and</code>	Verdadero si ambas condiciones se cumplen
	<code>or</code>	Verdadero si al menos una condición se cumple
	<code>not</code>	Invierte el valor booleano (<code>not True</code> → <code>False</code>)

Cortocircuito Lógico: En un `and`, si la primera condición es `False`, la segunda no se evalúa. En un `or`, si la primera condición es `True`, la segunda no se evalúa.

03 Veracidad (Truthy / Falsy) y Valor Nulo (None)

Evaluaciones a Falso (Falsy)

- El propio booleano `False`.
- El objeto nulo `None`.
- El número cero: `0` o `0.0`.
- Colecciones vacías: `''`, `[]`, `()`, `{}`, `set()`.

Identidad y Valor Nulo

`None` representa la ausencia de valor. Se recomienda comprobar valores nulos usando el operador `is` o `is not`.

Diferencia: `is` compara si dos variables apuntan al mismo espacio en memoria (`id(a) == id(b)`).

04 Sentencias Avanzadas de Control

A. Sentencia `match-case` (Pattern Matching)

Introducida en Python 3.10, permite comparar valores e inspeccionar estructuras de datos:

- **Guión bajo** (`_`): Actúa como comodín/patrón por defecto (`default`).
- **Desestructuración:** Extrae elementos de listas o tuplas según sus posiciones (ej. `case (int(x), int(y)):`).
- **Cláusulas Guarda:** Añaden filtros condicionales adicionales mediante `if` dentro del `case`.

B. Operador Morsa (`:=`)

Introducido en Python 3.8, permite **asignar un valor a una variable dentro de una expresión** condicional o de control.

05 Bucles y Sentencias de Repetición

Sentencia `while`

Ejecuta un bloque de código **mientras se cumpla una condición**.

- `break`: Rompe la ejecución del bucle.
- `continue`: Salta a la siguiente iteración.
- `else`: Se ejecuta si el bucle finaliza sin alcanzar un `break`.

Sentencia `for`

Recorre tipos de datos e infraestructuras que sean **iterables** (cadenas, listas, diccionarios, rangos).

Uso de `_`: Variable de control cuando no se necesita utilizar el valor iterado.

La Función `range()`

Genera secuencias numéricas bajo demanda sin cargarlas completamente en memoria.

```
range(start, stop, step)
```

- `start`: Valor inicial (opcional, por defecto `0`).
- `stop`: Límite superior **excluido** (llega hasta `stop - 1`).
- `step`: Paso o incremento (opcional, por defecto `1`).